

■ Lab 13 — Workspaces & Isolation du State

■ Objectifs du Lab

À la fin de ce lab, vous aurez :

- Créé des **workspaces** dev et prod et déployé sur chaque environnement
- Observé l'**isolation des state files** par workspace dans S3
- Utilisé `terraform.workspace` dans le code pour adapter la configuration
- Comparé l'approche workspaces avec l'approche **répertoires séparés**
- Compris quand utiliser l'une ou l'autre dans un contexte professionnel

■ **Workspaces vs Répertoires séparés** : Les workspaces conviennent aux environnements éphémères identiques (feature branches). Pour dev/prod qui divergent ou pour la production, préférez des **répertoires séparés** — l'isolation est totale et le risque d'erreur humaine est bien moindre.

■ Partie A — Workspaces

■ Étape 1 — Créer les fichiers

```
cd labs/lab13-workspaces
```

``backend.tf``

```
terraform {
  backend "s3" {
    bucket      = "tf-training-<votre-prenom>-982908300187"
    key         = "terraform.tfstate"
    region      = "eu-west-1"
    use_lockfile = true
    encrypt     = true
  }
}
```

■■ Remplacez `<votre-prenom>` par votre prénom. Ex : `tf-training-alice-982908300187`

``versions.tf``

```
terraform {
  required_version = ">= 1.6.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

`provider.tf`

```
provider "aws" {
  region = var.region
}
```

`variables.tf`

```
variable "username" {
  description = "Votre prenom"
  type        = string
}

variable "region" {
  description = "Region AWS"
  type        = string
  default     = "eu-west-1"
}
```

`locals.tf`

```
locals {
  # terraform.workspace retourne le nom du workspace courant
  environment = terraform.workspace

  prefix = "lab13-${var.username}-${local.environment}"

  # Adapter l'instance selon le workspace
  instance_type = local.environment == "prod" ? "t2.small" : "t2.micro"

  common_tags = {
    Lab           = "lab13"
    Username      = var.username
    Environment    = local.environment
    Workspace     = terraform.workspace
    ManagedBy     = "Terraform"
  }
}
```

`main.tf`

```

data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name     = "name"
    values   = ["al2023-ami-*-x86_64"]
  }
}

resource "aws_security_group" "lab13_sg" {
  name          = "${local.prefix}-sg"
  description   = "Security group Lab 13 - ${var.username} - ${local.environment}"

  tags = merge(local.common_tags, {
    Name = "${local.prefix}-sg"
  })
}

resource "aws_instance" "lab13_instance" {
  ami                  = data.aws_ami.amazon_linux.id
  instance_type        = local.instance_type
  vpc_security_group_ids = [aws_security_group.lab13_sg.id]

  tags = merge(local.common_tags, {
    Name = "${local.prefix}-instance"
  })
}

```

`outputs.tf`

```

output "workspace" {
  description = "Workspace courant"
  value       = terraform.workspace
}

output "instance_id" {
  description = "ID de l'instance"
  value       = aws_instance.lab13_instance.id
}

output "instance_type_used" {
  description = "Type d'instance utilisé"
  value       = local.instance_type
}

```

■ Étape 2 — Créer les workspaces

```
terraform init

# Lister les workspaces (default existe toujours)
terraform workspace list

# Créer et basculer sur le workspace dev
terraform workspace new dev

# Créer le workspace prod
terraform workspace new prod

# Lister les workspaces
terraform workspace list
```

■ Étape 3 — Déployer en dev

```
# Basculer sur dev
terraform workspace select dev

# Vérifier le workspace courant
terraform workspace show

# Déployer
terraform apply -var="username=<votre-prenom>"
terraform output workspace
terraform output instance_type_used
```

Résultat attendu :

```
workspace          = "dev"
instance_type_used = "t2.micro"
```

■ Étape 4 — Déployer en prod

```
terraform workspace select prod
terraform apply -var="username=<votre-prenom>"
terraform output workspace
terraform output instance_type_used
```

Résultat attendu :

```
workspace          = "prod"
instance_type_used = "t2.small"
```

■ Étape 5 — Observer les state files dans S3

```
# Les workspaces créent des dossiers séparés dans S3
aws s3 ls s3://tf-training-<votre-prenom>-982908300187/env:/
```

Résultat attendu :

```
env:/dev/terraform.tfstate
env:/prod/terraform.tfstate
```

■ Terraform stocke les states des workspaces non-default dans `env:/<workspace>/terraform.tfstate`. Le workspace default utilise `terraform.tfstate` à la racine.

Vérifier l'isolation — les ressources prod n'apparaissent pas dans le state dev :

```
terraform workspace select dev
terraform state list
# → uniquement les ressources dev

terraform workspace select prod
terraform state list
# → uniquement les ressources prod
```

■ Partie B — Comparaison avec les répertoires séparés

Dans un projet réel, les environnements dev et prod ont souvent des configurations très différentes (VPC, IAM, taille des ressources...). Dans ce cas, l'approche **répertoires séparés** est recommandée par HashiCorp.

Structure recommandée :

```
infra/
├── dev/
│   ├── backend.tf      ← key = "dev/terraform.tfstate"
│   ├── main.tf
│   ├── variables.tf
│   └── terraform.tfvars ← instance_type = "t2.micro"
└── prod/
    ├── backend.tf      ← key = "prod/terraform.tfstate"
    ├── main.tf
    ├── variables.tf
    └── terraform.tfvars ← instance_type = "t2.small"
```

Chaque répertoire est un projet Terraform totalement indépendant :

```
cd infra/dev && terraform init && terraform apply
cd infra/prod && terraform init && terraform apply
```

■ Un `terraform destroy` mal exécuté dans un workspace peut détruire la production. Avec des répertoires séparés, l'opérateur doit explicitement se positionner dans le bon dossier — une protection naturelle contre les erreurs humaines.

■ Étape 6 — Nettoyage

```
# Détruire prod
terraform workspace select prod
terraform destroy -var="username=<votre-prenom>"

# Détruire dev
terraform workspace select dev
terraform destroy -var="username=<votre-prenom>"

# Revenir sur default et supprimer les workspaces
terraform workspace select default
terraform workspace delete dev
terraform workspace delete prod
```

■ Validation du Lab

#	Critère	Validé
1	Workspaces dev et prod créés	■
2	terraform.workspace retourne le bon nom dans chaque workspace	■
3	instance_type_used = t2.micro en dev	■
4	instance_type_used = t2.small en prod	■
5	State files séparés dans S3 env:/dev/ et env:/prod/	■
6	terraform state list ne montre que les ressources du workspace actif	■
7	terraform destroy supprime les ressources sur chaque workspace	■

■ Résolution des problèmes courants

Problème	Cause	Solution
Ressources prod visibles dans dev	Mauvais workspace sélectionné	terraform workspace show pour vérifier
Error: workspace already exists	Workspace déjà créé	Utiliser terraform workspace select dev
State file non trouvé dans S3	Backend pas initialisé	terraform init dans le bon dossier

■ Fin du Lab 13 — Vous gérez plusieurs environnements avec les workspaces !

> Le Lab 14 introduit **terraform import** pour gérer des ressources créées hors Terraform.